Rapport detaille - Marketplace

Releases 1 a 4, Sprints 1 a 9

Periode cible: Janvier 2025 - Juin 2025
Contexte: Frontend Vue 3, Backend NestJS, MySQL, JWT, Docker

---

1) Vue d'ensemble

1.1 Use Case general (systeme global)

Acteurs:

- Visiteur
- Client
- Vendeur
- Administrateur
- Service Paiement (externe)
- Service Notification (email/push)
- Service IA

Use cases majeurs:

- UC-01 S'inscrire
- UC-02 Se connecter
- UC-03 Gerer profil
- UC-04 Gerer utilisateurs (admin)
- UC-05 Gerer produits
- UC-06 Gerer categories
- UC-07 Rechercher produits
- UC-08 Gerer panier
- UC-09 Passer commande
- UC-10 Payer commande
- UC-11 Suivre commande
- UC-12 Gerer dashboards
- UC-13 Commenter / Noter
- UC-14 Suivre vendeur
- UC-15 Recommandations IA / Chatbot / Analyse avis

1.2 Diagramme de classes general (domaine)

Classes principales:

- User(id, nom, email, motDePasseHash, role, statut)
- Profile(userId, adresse, telephone, avatar, preferences)
- Category(id, nom, description, parentId)
- Product(id, sellerId, categoryId, nom, description, prix, stock, statut)
- Cart(id, userId)
- CartItem(id, cartId, productId, quantite, prixUnitaire)
- Order(id, userId, total, statut, adresseLivraison, dateCommande)
- OrderItem(id, orderId, productId, quantite, prixUnitaire)
- Payment(id, orderId, methode, transactionId, statut, montant)

- Review(id, userId, productId, note, commentaire, createdAt)
- SellerFollow(id, userId, sellerId, createdAt)
- Notification(id, userId, type, message, lu, createdAt)
- Recommendation(id, userId, productId, score, source)
- ChatSession(id, userId, contexte, createdAt)

Relations:

- User 1..1 Profile
- User 1..* Order
- User 1..1 Cart
- Cart 1..* CartItem
- Product 1..* CartItem
- Order 1..* OrderItem
- Product 1..* OrderItem
- User(seller) 1..* Product
- Category 1..* Product
- User 1..* Review
- Product 1..* Review
- User(client) .. User(vendeur) via SellerFollow
- Order 1..1 Payment
- User 1..* Notification

---

2) Release 1 - Gestion des utilisateurs

Objectif: Mettre en place l'acces a la plateforme

Sprint 1 - Authentification

A. Use Case general sprint

- UC-S1-01 Creation de compte
- UC-S1-02 Connexion

B. Use Case raffine sprint

- UC-S1-01 inclut:
  - Valider formulaire
  - Verifier unicite email
  - Hasher mot de passe
  - Creer compte
- UC-S1-02 inclut:
  - Valider credentials
  - Verifier role
  - Generer JWT
  - Rediriger vers espace role
- UC-S1-02 etend:
  - Recuperer mot de passe (si echec repetitif / oubli)

C. Diagramme de classes sprint (focus)

- AuthService(login, register, forgotPassword, resetPassword)

- UserRepository
- JwtStrategy
- RoleGuard
- SignInDTO
- SignUpDTO
- ResetPasswordDTO

D. Diagramme de scenario (UC principal: Connexion)

Sequence:

1. Utilisateur -> UI SignIn: saisir email+mot de passe
2. UI -> AuthController: POST /auth/login
3. AuthController -> AuthService: login(dto)
4. AuthService -> UserRepository: findByEmail(email)
5. AuthService -> PasswordHasher: compare(hash, plain)
6. AuthService -> JwtService: sign(payload role/id)
7. AuthService -> AuthController: token + user role
8. AuthController -> UI: 200 OK
9. UI -> RouterGuard: verifier role + route
10. RouterGuard -> Dashboard adapte

E. Description du scenario UC-S1-02 Connexion

- Acteur principal: Utilisateur inscrit
- Precondition: Compte actif et non bloque
- Declencheur: Clic sur "Se connecter"
- Flux nominal:
    1. Saisie credentials
    2. Validation format
    3. Authentification backend
    4. Emission JWT
    5. Redirection selon role
- Flux alternatifs:
    - A1: mauvais mot de passe -> message erreur + compteur tentative
    - A2: compte inexistant -> proposer inscription
    - A3: compte bloque -> contact support
- Postcondition: Session authentifiee et acces autorise

Sprint 2 - Gestion des profils

A. Use Case general sprint

- UC-S2-01 Modifier profil
- UC-S2-02 Gestion des utilisateurs (admin)

B. Use Case raffine sprint

- UC-S2-01 inclut:
    - Consulter profil
    - Modifier infos personnelles
    - Upload avatar
    - Sauvegarder changements

- UC-S2-02 inclut:
  - Lister utilisateurs
  - Rechercher un utilisateur
  - Changer role/statut
  - Activer/Desactiver compte
  - Consulter details utilisateur

C. Diagramme de classes sprint (focus)

- ProfileController / ProfileService
- UserController / UserService
- AdminUserManagementService
- UpdateProfileDTO
- UpdateUserRoleDTO
- UserEntity
- ProfileEntity

D. Diagramme de scenario (UC principal: Gestion des utilisateurs admin)

Sequence:

1. Admin -> UI AdminUsers: ouvrir liste utilisateurs
2. UI -> UserController: GET /users
3. UserController -> UserService: findAll(filter)
4. UserService -> UserRepository: query with pagination
5. Admin -> UI: changer role/statut
6. UI -> UserController: PATCH /users/:id
7. UserController -> AdminUserManagementService: updateRoleStatus()
8. Service -> UserRepository: save()
9. Service -> NotificationService: notifier utilisateur (optionnel)
10. UI: afficher confirmation

E. Description du scenario UC-S2-02 Gestion utilisateurs

- Acteur principal: Administrateur
- Precondition: Admin authentifie
- Declencheur: Besoin de modifier un compte
- Flux nominal:
  1. Consultation liste
  2. Selection utilisateur
  3. Modification role/statut
  4. Validation
  5. Sauvegarde + confirmation
- Flux alternatifs:
  - A1: utilisateur non trouve -> erreur 404
  - A2: role invalide -> erreur validation
  - A3: conflit de permission -> refus action
- Postcondition: Compte mis a jour et trace d'audit conservee

---

3) Release 2 - Gestion des produits et categories

Objectif: Construire le catalogue marketplace

Sprint 3 - Gestion des produits

A. Use Case general sprint

- UC-S3-01 Ajouter produit
- UC-S3-02 Modifier produit
- UC-S3-03 Supprimer produit

B. Use Case raffine sprint

- UC-S3-01 inclut:
  - Saisir informations produit
  - Importer images
  - Associer categorie
  - Definir stock/prix
  - Publier
- UC-S3-02 inclut:
  - Charger fiche produit
  - Editer attributs
  - Verifier proprietaire vendeur
  - Sauvegarder nouvelle version
- UC-S3-03 inclut:
  - Verification dependances commande
  - Soft delete produit

C. Diagramme de classes sprint (focus)

- ProductController / ProductService
- ProductEntity
- CreateProductDTO / UpdateProductDTO
- UploadService
- CategoryEntity
- SellerPolicyGuard

D. Diagramme de scenario (UC principal: Ajouter produit)

Sequence:

1. Vendeur -> UI AddProduct: saisir donnees
2. UI -> UploadController: upload image
3. UploadController -> UploadService: store(file)
4. UI -> ProductController: POST /products
5. ProductController -> ProductService: create(dto, sellerId)
6. ProductService -> CategoryRepository: verify(category)
7. ProductService -> ProductRepository: save(product)
8. ProductController -> UI: 201 created
9. UI -> ProductsList: rafraichir liste

E. Description du scenario UC-S3-01 Ajouter produit

- Acteur principal: Vendeur
- Precondition: Vendeur authentifie
- Declencheur: Clic "Ajouter produit"
- Flux nominal:

    1. Saisie informations
    2. Upload media
    3. Validation metier
    4. Enregistrement
    5. Publication dans catalogue
- Flux alternatifs:
    - A1: categorie invalide
    - A2: image non conforme
    - A3: prix/stock invalides
- Postcondition: Produit cree et visible selon statut

Sprint 4 - Organisation et recherche

A. Use Case general sprint

- UC-S4-01 Gestion des categories
- UC-S4-02 Recherche de produits

B. Use Case raffine sprint

- UC-S4-01 inclut:
    - Creer categorie
    - Modifier categorie
    - Supprimer categorie
    - Organiser hierarchie parent/enfant
- UC-S4-02 inclut:
    - Rechercher par mot-cle
    - Filtrer par categorie/prix
    - Trier resultats
    - Paginer resultats

C. Diagramme de classes sprint (focus)

- CategoryController / CategoryService
- CategoryEntity(parentId)
- CatalogController / SearchService
- ProductSearchQueryDTO
- ProductReadModel

D. Diagramme de scenario (UC principal: Recherche produit)

Sequence:

1. Client -> UI Catalog: saisir mot-cle + filtres
2. UI -> CatalogController: GET /products/search
3. CatalogController -> SearchService: search(query)
4. SearchService -> ProductRepository: apply filters/sort/pagination
5. SearchService -> CatalogController: result page
6. CatalogController -> UI: liste produits
7. Client -> UI: ouvrir fiche produit

E. Description du scenario UC-S4-02 Recherche produit

- Acteur principal: Client (ou visiteur)
- Precondition: Catalogue disponible
- Declencheur: Recherche depuis barre catalogue
- Flux nominal:
    1. Saisie critere
    2. Excecution recherche
    3. Affichage resultats
    4. Affinage filtres
    5. Selection produit
- Flux alternatifs:
    - A1: aucun resultat -> suggestions categories
    - A2: filtre incoherent -> reset partiel
- Postcondition: Resultat pertinent affiche pour consultation

---

4) Release 3 - Gestion des commandes et tableaux de bord

Objectif: Implementer achat et suivi

Sprint 5 - Commande

A. Use Case general sprint

- UC-S5-01 Ajouter au panier
- UC-S5-02 Passer commande

B. Use Case raffine sprint

- UC-S5-01 inclut:
    - Verifier stock
    - Ajouter ligne panier
    - Recalculer total
- UC-S5-02 inclut:
    - Verifier panier non vide
    - Confirmer adresse
    - Creer commande + lignes
    - Initialiser statut "en attente paiement"

C. Diagramme de classes sprint (focus)

- CartController / CartService
- CartEntity / CartItemEntity
- OrderController / OrderService
- OrderEntity / OrderItemEntity
- CreateOrderDTO / AddToCartDTO

D. Diagramme de scenario (UC principal: Passer commande)

Sequence:

1. Client -> UI Checkout: confirmer panier+adresse
2. UI -> OrderController: POST /orders
3. OrderController -> OrderService: createFromCart(userId)

4. OrderService -> CartService: getCurrentCart()
5. OrderService -> StockService: reserve(items)
6. OrderService -> OrderRepository: save(order)
7. OrderService -> CartService: clearCart()
8. OrderController -> UI: orderId + statut

E. Description du scenario UC-S5-02 Passer commande

- Acteur principal: Client
- Precondition: Panier valide avec stock disponible
- Declencheur: Clic "Passer commande"
- Flux nominal:
    1. Validation donnees checkout
    2. Creation commande
    3. Reservation stock
    4. Confirmation commande
- Flux alternatifs:
    - A1: stock insuffisant -> retour panier
    - A2: adresse invalide -> blocage validation
- Postcondition: Commande creee en attente paiement

Sprint 6 - Paiement et suivi

A. Use Case general sprint

- UC-S6-01 Paiement securise
- UC-S6-02 Suivi des commandes
- UC-S6-03 Notification vendeur

B. Use Case raffine sprint

- UC-S6-01 inclut:
    - Creer intention paiement
    - Redirection passerelle
    - Confirmer transaction
    - Mettre a jour statut commande
- UC-S6-02 inclut:
    - Consulter timeline commande
    - Etat: en preparation / expediee / livree
- UC-S6-03 inclut:
    - Notifier vendeur nouvelle commande
    - Notifier vendeur paiement confirme

C. Diagramme de classes sprint (focus)

- PaymentController / PaymentService
- PaymentEntity
- PaymentGatewayAdapter
- OrderTrackingService
- NotificationService
- MailService

D. Diagramme de scenario (UC principal: Paiement securise)

Sequence:

1. Client -> UI Payment: choisir methode
2. UI -> PaymentController: POST /payments/intent
3. PaymentController -> PaymentService: createIntent(orderId)
4. PaymentService -> PaymentGatewayAdapter: init(amount)
5. Gateway -> PaymentService: transactionId + url
6. Client -> Gateway: valider paiement
7. Gateway -> WebhookController: callback success
8. WebhookController -> PaymentService: confirm(transactionId)
9. PaymentService -> OrderService: setStatus(PAID)
10. PaymentService -> NotificationService: notifier vendeur/client

E. Description du scenario UC-S6-01 Paiement securise

- Acteur principal: Client
- Acteur secondaire: Service Paiement
- Precondition: Commande creee en attente de paiement
- Declencheur: Validation paiement
- Flux nominal:
    1. Creation intention
    2. Validation paiement chez PSP
    3. Callback de confirmation
    4. Maj commande payee
    5. Envoi notifications
- Flux alternatifs:
    - A1: paiement refuse
    - A2: timeout passerelle
    - A3: callback invalide (signature)
- Postcondition: Paiement trace et commande synchronisee

Sprint 7 - Tableaux de bord

A. Use Case general sprint

- UC-S7-01 Dashboard admin
- UC-S7-02 Dashboard vendeur
- UC-S7-03 Historique client

B. Use Case raffine sprint

- UC-S7-01 inclut:
    - KPI utilisateurs / ventes / commandes / litiges
    - Alertes moderation
- UC-S7-02 inclut:
    - KPI chiffre d'affaires vendeur
    - Etat commandes vendeur
    - Performance produits
- UC-S7-03 inclut:
    - Historique commandes client

    - Details commandes et statuts

C. Diagramme de classes sprint (focus)

- DashboardController / DashboardService
- AdminMetricsService
- SellerMetricsService
- CustomerHistoryService
- AggregationQueryBuilder

D. Diagramme de scenario (UC principal: Dashboard vendeur)

Sequence:

1. Vendeur -> UI SellerDashboard
2. UI -> DashboardController: GET /dashboard/seller
3. Controller -> SellerMetricsService: buildMetrics(sellerId)
4. Service -> OrderRepository/ProductRepository: aggregate()
5. Service -> DashboardController: metrics DTO
6. Controller -> UI: KPI + graphiques

E. Description du scenario UC-S7-02 Dashboard vendeur

- Acteur principal: Vendeur
- Precondition: Vendeur authentifie
- Declencheur: Ouverture dashboard
- Flux nominal:
    1. Chargement donnees
    2. Aggregation metriques
    3. Affichage KPI
- Flux alternatifs:
    - A1: aucune donnee -> etat vide
    - A2: erreur API -> message retry
- Postcondition: Vendeur dispose d'une vue pilotage

---

5) Release 4 - Interactions et intelligence artificielle

Objectif: Ameliorer l'experience utilisateur

Sprint 8 - Interaction

A. Use Case general sprint

- UC-S8-01 Commentaires et notes
- UC-S8-02 Reponse aux commentaires
- UC-S8-03 Suivre vendeur

B. Use Case raffine sprint

- UC-S8-01 inclut:
    - Verifier achat effectif (regle optionnelle)
    - Publier note/commentaire
    - Editer ou supprimer avis

- UC-S8-02 inclut:
  - Vendeur consulte avis
  - Repond a un avis
  - Notification client
- UC-S8-03 inclut:
  - Suivre vendeur
  - Se desabonner
  - Recevoir actualites vendeur

C. Diagramme de classes sprint (focus)

- ReviewController / ReviewService / ReviewEntity
- ReviewReplyEntity
- SellerFollowController / SellerFollowService / SellerFollowEntity
- EngagementNotificationService

D. Diagramme de scenario (UC principal: Reponse commentaire)

Sequence:

1. Vendeur -> UI AvisRecus: ouvrir avis produit
2. UI -> ReviewController: GET /reviews/product/:id
3. Vendeur -> UI: saisir reponse
4. UI -> ReviewController: POST /reviews/:id/reply
5. ReviewController -> ReviewService: addReply()
6. ReviewService -> ReviewRepository: save reply
7. ReviewService -> NotificationService: notifier client
8. UI: confirmation

E. Description du scenario UC-S8-02 Reponse commentaire

- Acteur principal: Vendeur
- Precondition: Avis existe et appartient a un produit du vendeur
- Declencheur: Clic "Repondre"
- Flux nominal:
  1. Lecture avis
  2. Saisie reponse
  3. Publication reponse
  4. Notification client
- Flux alternatifs:
  - A1: avis supprime entre-temps
  - A2: contenu non conforme (moderation)
- Postcondition: Reponse reliee a l'avis et visible client

Sprint 9 - Intelligence artificielle

A. Use Case general sprint

- UC-S9-01 Recommandations produits
- UC-S9-02 Chatbot
- UC-S9-03 Analyse des avis

B. Use Case raffine sprint

- UC-S9-01 inclut:
  - Collecte historique/navigation
  - Scoring candidats
  - Exposition top-N recommandations
- UC-S9-02 inclut:
  - Saisie question utilisateur
  - Recherche contexte catalogue
  - Generation reponse
- UC-S9-03 inclut:
  - Classification sentiment
  - Resume des themes
  - Alertes qualite vendeur/produit

C. Diagramme de classes sprint (focus)

- RecommendationService
- EmbeddingService
- ChatController / ChatService
- ReviewAnalyticsService
- IAProviderAdapter
- RecommendationDTO / ChatRequestDTO / SentimentSummaryDTO

D. Diagramme de scenario (UC principal: Chatbot)

Sequence:

1. Client -> UI ChatWidget: poser question
2. UI -> ChatController: POST /chat
3. ChatController -> ChatService: answer(question, userContext)
4. ChatService -> RetrievalService: fetch relevant products/reviews
5. ChatService -> IAProviderAdapter: generate(prompt+context)
6. IAProviderAdapter -> ChatService: response text
7. ChatService -> ChatController: response + suggestions
8. ChatController -> UI: afficher reponse

E. Description du scenario UC-S9-02 Chatbot

- Acteur principal: Client
- Acteur secondaire: Service IA
- Precondition: Widget chat disponible
- Declencheur: Envoi question
- Flux nominal:
  1. Reception question
  2. Construction contexte metier
  3. Generation IA
  4. Restitution reponse + recommandations
- Flux alternatifs:
  - A1: service IA indisponible -> fallback FAQ
  - A2: question ambigue -> demande precision
- Postcondition: Reponse contextualisee fournie

---

6) Matrice de tracabilite (Release -> Sprint -> UC principal)

Release	Sprint	Theme	UC principal de scenario
R1	S1	Authentification	UC-S1-02 Connexion
R1	S2	Profils & Admin users	UC-S2-02 Gestion utilisateurs
R2	S3	Produits	UC-S3-01 Ajouter produit
R2	S4	Categories & Recherche	UC-S4-02 Recherche produit
R3	S5	Commande	UC-S5-02 Passer commande
R3	S6	Paiement & Suivi	UC-S6-01 Paiement securise
R3	S7	Dashboards	UC-S7-02 Dashboard vendeur
R4	S8	Interactions	UC-S8-02 Reponse commentaire
R4	S9	Intelligence artificielle	UC-S9-02 Chatbot

---

7) Organisation des livrables UML conseilles

Arborescence recommandee:

- `docs/uml/release-1/sprint-1/`
- `docs/uml/release-1/sprint-2/`
- `docs/uml/release-2/sprint-3/`
- `docs/uml/release-2/sprint-4/`
- `docs/uml/release-3/sprint-5/`
- `docs/uml/release-3/sprint-6/`
- `docs/uml/release-3/sprint-7/`
- `docs/uml/release-4/sprint-8/`
- `docs/uml/release-4/sprint-9/`

Types de fichiers a y placer:

- `usecase-general.puml`
- `usecase-rafine.puml`
- `class-diagram.puml`
- `sequence-scenario.puml`
- `description-scenario-uc.md`

---

8) Conclusion

Ce rapport fournit une base complete, alignee avec votre decoupage Release 1 a 4 / Sprint 1 a 9, pour un dossier academique ou professionnel.
Il couvre:

- diagramme de classe general,
- use case general et raffine (global + par sprint),
- diagramme de scenario par sprint,

- description textuelle du scenario UC principal par sprint.